

APIs Java: tratamento de exceções

CST em Análise e Desenvolvimento de Sistemas

Prof. Emerson Ribeiro de Mello

mello@ifsc.edu.br

Licenciamento



Slides licenciados sob [Creative Commons "Atribuição 4.0 Internacional"](#)

Tratamento de exceções

Um simples programa Java

```
import java.util.Scanner;

public class Principal{

    public static void main(String[] args){
        Scanner teclado = new Scanner(System.in);

        int[] vetor = new int[10];

        System.out.print("Entre com o número: ");
        int numero = teclado.nextInt();

        System.out.print("Em qual posição ficará?: ");
        int posicao = teclado.nextInt();

        vetor[posicao] = numero;
    }
}
```

- O código acima é seguro? Executará sempre sem problemas?

Tratamento de exceções

Exceção

Evento que ocorre durante a execução do programa e que foge do fluxo normal do programa

Tratamento de exceções

Exceção

Evento que ocorre durante a execução do programa e que foge do fluxo normal do programa

Tratamento de exceções

Permite ao programa **capturar** e **tratar exceções** em vez de deixá-los ocorrer e assim sofrer com as consequências

Tratamento de exceções

Exceção

Evento que ocorre durante a execução do programa e que foge do fluxo normal do programa

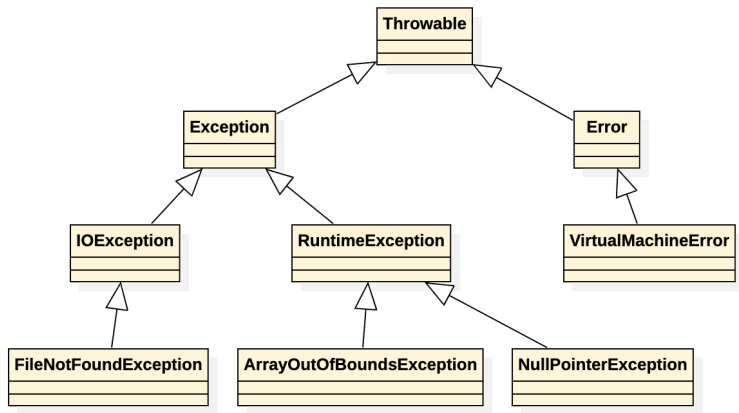
Tratamento de exceções

Permite ao programa **capturar** e **tratar exceções** em vez de deixá-los ocorrer e assim sofrer com as consequências

- 1 Adequado para situações em que o programador não tem controle, como comunicação com banco de dados, arquivos, rede, entrada de dados, etc.
- 2 Adequado para separar o tratamento do fluxo normal do programa

Hierarquia de exceções em Java

- Quando um evento inesperado ocorre é criado um **objeto de exceção**
 - contém informações sobre o erro como tipo, mensagem e estado do programa quando o erro ocorreu



Hierarquia parcial. Fonte: <https://www.oracle.com/br/technical-resources/article/java/erros-java-exceptions.html>

Classificação das exceções

- **Exceções verificadas** (*checked exceptions*)
 - O compilador obriga o programador a tratar a exceção
 - `Exception` e suas subclasses, exceto `RuntimeException` e suas subclasses
 - Exemplos: `IOException`, `FileNotFoundException`, `SQLException`

Classificação das exceções

■ Exceções verificadas (*checked exceptions*)

- O compilador obriga o programador a tratar a exceção
- Exception e suas subclasses, exceto RuntimeException e suas subclasses
 - Exemplos: IOException, FileNotFoundException, SQLException

■ Exceções não verificadas (*unchecked exceptions*)

- O compilador não obriga o programador a tratar a exceção
 - Pode resultar em falhas de execução quando não tratadas
- RuntimeException e suas subclasses
 - Exemplos: ArithmeticException, NullPointerException, ArrayIndexOutOfBoundsException

Bloco try...catch para tratar exceções

- O bloco **try** é usado para envolver o código que pode lançar uma exceção
- O bloco **catch** é usado para capturar e tratar a exceção
 - Se ocorrer uma exceção dentro do bloco **try**, o fluxo de execução passa automaticamente para um bloco **catch**
 - Se não ocorrer exceção, então o fluxo de execução passa para a próxima linha após os blocos **catch**

```
System.out.println("Iniciando o programa");  
try{  
    // instruções que possam vir a disparar uma exceção  
}catch(Tipo da exceção){  
    // instruções para lidar com a exceção gerada  
}  
System.out.println("continuando o programa");
```

Exemplo 1: Tipo misturado (int vs String)

```
public static void main(String[] args){
    Scanner ler = new Scanner(System.in);
    int a, b;

    try{
        System.out.print("Entre com o número: ");
        a = ler.nextInt();
        System.out.print("Entre com o número: ");
        b = ler.nextInt();

        int res = a / b;

        System.out.println(a + " dividido por " + b + " = " + res);
    }catch(InputMismatchException e){
        System.err.println("Só é permitido números inteiros");
        ler.nextLine(); // limpa o buffer do teclado
    }
    System.out.println("Fim do programa");
}
```

Divisão por zero em Java

<https://docs.oracle.com/javase/specs/jls/se14/html/jls-15.html#jls-15.17.2>

Inteiros e números reais são representados de maneira diferente na memória

- Dividir número inteiro por zero resultará em uma exceção do tipo `ArithmeticException`
 - $10/0 \rightarrow \text{ArithmeticException}$
- Dividir tipos reais (*float* ou *double*) seguem o Padrão IEEE 754 para Aritmética Binária de Ponto Flutuante
 - $0.0/0 \rightarrow \text{Not a Number (NaN)}$
 - $10.0/0 \rightarrow \text{POSITIVE_INFINITY}$
 - $-1.0/0 \rightarrow \text{NEGATIVE_INFINITY}$

Determinando o tipo da exceção

- Para cada bloco **try** é possível ter um ou mais blocos **catch**
- Cada bloco **catch** é responsável por tratar um tipo específico de exceção

Cadeia de blocos catch

Organizar os blocos **catch** de forma que as exceções mais específicas sejam capturadas antes das exceções mais genéricas

Exemplo 2: Cadeia de exceções com catch

```
public static void main(String[] args){
    int[] numeros = new int[10];
    Scanner teclado = new Scanner(System.in);
    try{
        System.out.print("Entre com o número: ");
        int numero = teclado.nextInt();
        System.out.print("Em qual posição ficará?: ");
        int posicao = teclado.nextInt();

        numeros[posicao] = numero;

    }catch(java.util.InputMismatchException e){
        System.err.println("Só é permitido números inteiros");
    }catch(java.lang.ArrayIndexOutOfBoundsException e){
        System.err.println("Estouro de limite de vetor");
    }catch(Exception e){
        System.err.println("Ocorreu o erro: " + e.toString());
    }
    System.out.println("Fim do programa");
}
```

Disparar, encaminhar e capturar exceções

■ **Disparar exceção** (*throw exception*)

- Usado para lançar uma exceção (palavra-chave `throw` seguida de um objeto de exceção)

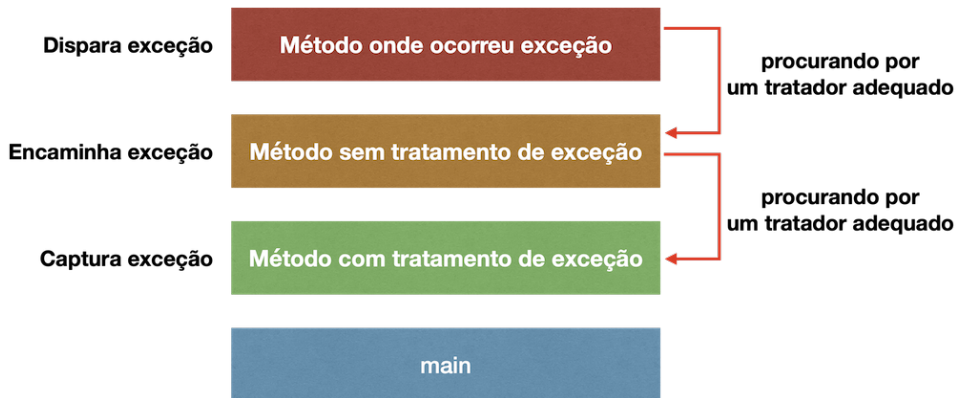
■ **Encaminhar exceção** (*forward exception*)

- Usado para passar a exceção para o método que o invocou (usando a palavra-chave `throws` no cabeçalho do método)

■ **Capturar exceção** (*catch exception*)

- Usado para tratar uma exceção que foi lançada (bloco `try...catch`)

Disparar, encaminhar e capturar exceções



Exemplo: tratando a exceção

Construtor da classe `MaskFormatter` dispara uma exceção do tipo `ParseException`

```
public static String formata(String mascara, String valor){
    MaskFormatter mask = null;
    String resultado = "";

    try {
        mask = new MaskFormatter(mascara); // dispara uma exceção verificada
        mask.setValueContainsLiteralCharacters(false);
        mask.setPlaceholderCharacter('_');
        resultado = mask.valueToString(valor);
    } catch (ParseException e) {
        System.err.println("Erro ao formatar a String");
    }
    return resultado;
}

public static void main(String[] args) {
    System.out.println(formata("(##) #####-####", "48998765432"));
}
```

Exemplo: encaminhando a exceção

Construtor da classe `MaskFormatter` dispara uma exceção do tipo `ParseException`

```
// encaminhando a exceção para o método que o invocou
public static String formata(String m, String v) throws ParseException {
    String resultado = "";
    MaskFormatter mask = new MaskFormatter(m); // dispara uma exceção verificada
    mask.setValueContainsLiteralCharacters(false);
    mask.setPlaceholderCharacter('_');
    resultado = mask.valueToString(v);
    return resultado;
}

public static void main(String[] args) {
    try {
        System.out.println(formata("(##) #####-####", "48998765432"));
    } catch (ParseException e) {
        System.err.println("Erro ao formatar a String");
    }
}
```

Exemplo: disparando uma exceção

Se o valor de um dos parâmetros for nulo

```
public static String formata(String m, String v) throws ParseException {
    if (m == null || v == null) {
        throw new IllegalArgumentException("Máscara e valor não podem ser nulos");
    }
    String resultado = "";
    MaskFormatter mask = new MaskFormatter(m);
    mask.setValueContainsLiteralCharacters(false);
    mask.setPlaceholderCharacter('_');
    resultado = mask.valueToString(v);
    return resultado;
}

public static void main(String[] args) {
    try {
        System.out.println(formata(null, "48998765432"));
    } catch (ParseException e) {
        System.err.println("Erro ao formatar a String");
    } catch (IllegalArgumentException e) {
        System.err.println("Erro: " + e.getMessage());
    }
}
```

Bloco finally

Local ideal para liberar recursos adquiridos

- Recursos que são abertos dentro de um bloco **try** podem ser fechados no bloco **finally**
 - Arquivos, conexões com banco de dados, sockets, etc.
- As linhas dentro de um bloco **catch** só serão executadas se for lançada uma exceção do tipo que o bloco está apto a capturar e tratar
- As linhas dentro do bloco **finally** sempre serão executadas, independente de ocorrer exceção ou não
- As linhas dentro do bloco **finally** sempre serão executadas, mesmo se houver instruções `return`, `continue` ou `break` dentro do bloco **try**

Exemplo: bloco finally

```
public int lerTeclado(){
    Scanner teclado = new Scanner(System.in);

    do{

        try{
            System.out.print("Entre com um número: ");
            int num = teclado.nextInt();
            return num; // se o número for lido corretamente, então retorne

        }catch(Exception e){
            System.err.println("Erro: " + e.toString());
        }finally{
            System.out.print("Sempre será executada");
        }

        System.out.print("Se o return for executado, então essa linha nunca será alcançada");

    }while(true);
}
```

Fechando recursos no bloco finally

```
public void lerArquivo(String nomeArquivo){  
    try{  
        Scanner leitor = new Scanner(new File(nomeArquivo));  
  
        while(leitor.hasNext()){  
            System.out.println(leitor.nextLine());  
        }  
  
    }catch(IOException e){  
        System.err.println("Erro ao ler o arquivo");  
  
    }finally{  
        if(leitor != null){  
            leitor.close();  
        }  
    }  
}
```

Dificuldades do bloco finally

- As instruções dentro do bloco finally também podem lançar exceções

```
// Conexão com banco de dados MySQL
public void conectar(){
    Connection conexao = null;
    try{
        conexao = DriverManager.getConnection("jdbc:mysql://localhost:3306/banco", "
        usuario", "senha");
        // instruções para manipular o banco de dados
    }catch(SQLException e){
        System.err.println("Erro ao conectar com o banco de dados");
    }finally{
        if(conexao != null){
            try{
                conexao.close();
            }catch(SQLException e){
                System.err.println("Erro ao fechar a conexão com o banco de dados");
            }
        }
    }
}
```


Try with resources

- É possível usar o bloco `try` para fechar recursos automaticamente sem a necessidade de usar o bloco `finally`
- Torna o código mais limpo e fácil de ler
- O recurso deve implementar a interface `AutoCloseable`
 - `FileInputStream`, `FileOutputStream`, `Scanner`, `Connection`, `Statement`, `ResultSet`, etc.

Fechando recursos no bloco try with resources

```
public void lerArquivo(String nomeArquivo){  
    try(Scanner leitor = new Scanner(new File(nomeArquivo))){  
        while(leitor.hasNext()){  
            System.out.println(leitor.nextLine());  
        }  
    }catch(IOException e){  
        System.err.println("Erro ao ler o arquivo");  
    }  
}
```

Fechando conexão com banco de dados

Usando o bloco try with resources

```
public void conectar(){  
    // É possível declarar vários recursos separados dentro do try  
    try(  
        Connection conexao = DriverManager.getConnection("jdbc:mysql://localhost:3306/  
banco", "usuario", "senha");  
        Statement stmt = conexao.createStatement()  
    ){  
  
        // instruções para manipular o banco de dados  
  
    }catch(SQLException e){  
        System.err.println("Erro ao conectar com o banco de dados");  
    }  
}
```

Leitura obrigatória

- HOSRTMANN, C. S., CORNELL, G. **Core Java** - 8^a Edição, 2010. Capítulo 11.
- <https://docs.oracle.com/javase/tutorial/essential/exceptions>